

Symmetric Key Cryptography

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Cryptography

- A tremendous tool enabling **secure communication** over **insecure communication channels**
- The **basis** for many security mechanisms
 - E.g., your online browsing is secured using cryptography (foundations to TLS)
 - **Not** the solution to all security problems
 - Reliable if implemented and used properly

Three Main Goals

- Confidentiality
 - Preventing adversaries from learning the content of our private data
- Integrity
 - Preventing adversaries from altering our data
- Authenticity
 - Ensuring that the data is created by the expected user
 - Generally implies/requires integrity

Special Guests

- Alice
- Bob
- Eve
 - The attacker
 - Eavesdropper
 - Malice
- Sometimes Charlie

Communication



- **Confidentiality**: Preventing Eve from learning the message
- **Integrity**: Preventing Eve from modifying the message
- **Authenticity**: Ensuring the message is from Alice

Terminologies

- Encryption
 - Converting plaintext into ciphertext
- Decryption
 - Converting ciphertext into plaintext
- Key
 - A randomly chosen value used in encryption and decryption
- Cryptoanalysis
 - Breaking the encryption/decryption scheme by analyzing the algorithm/implementation

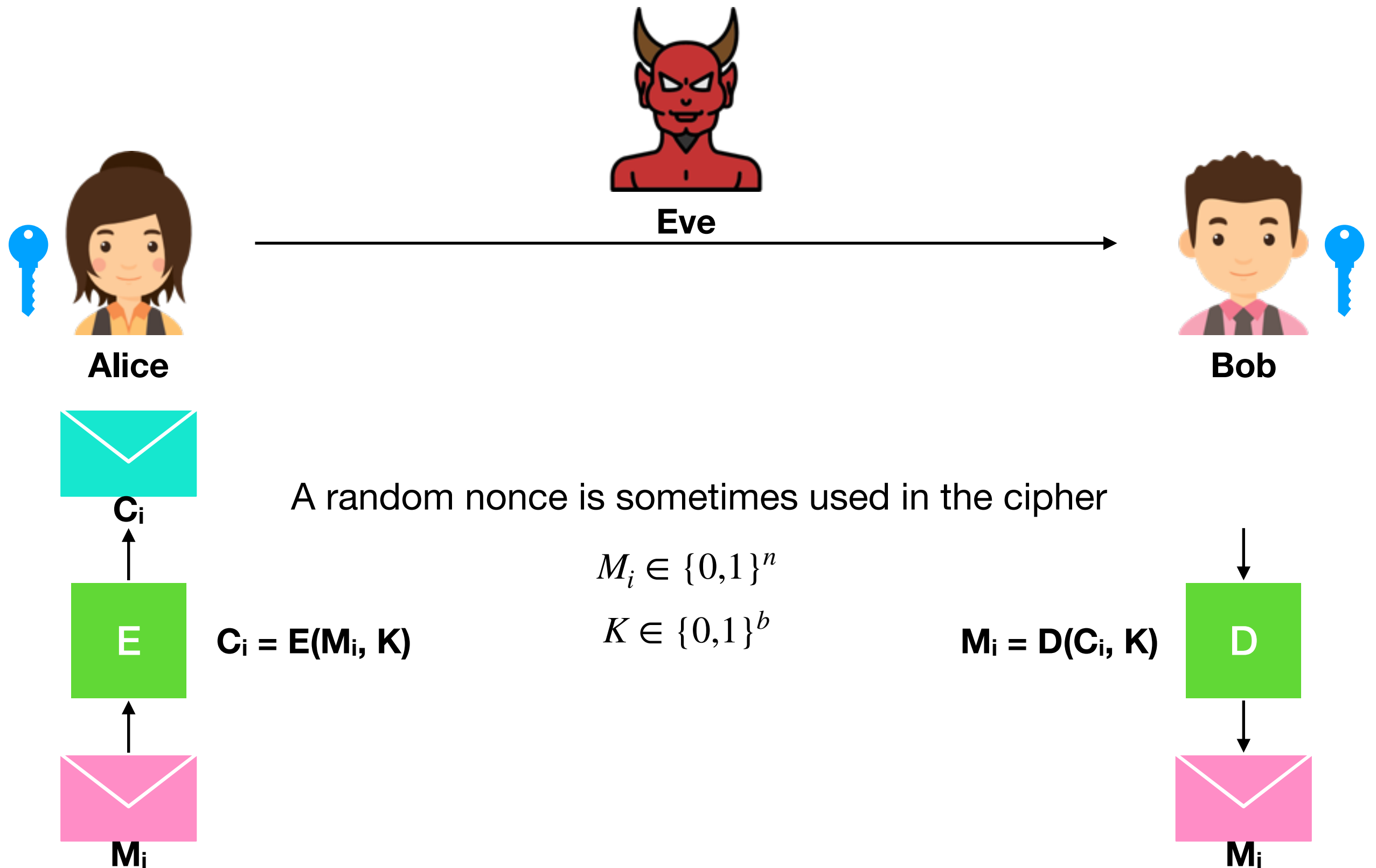
Crypto Algorithms

- Symmetric algorithms
 - Single and secret key
 - The encryption and decryption keys are the same
- Asymmetric algorithms
 - A pair of keys (One is usually public)
 - Public-key algorithms
 - The data encrypted with one key can be decrypted with the other key
 - It is computationally impractical to calculate the other key given one key

Crypto Algorithms (Cont.)

- One-way functions: It is difficult to inverse the computation
- One-way hash functions
 - Generate fixed-length output given any input
 - The output looks random
 - Collision resistance
 - Pre-image resistance
 - Second pre-image resistance
 - Can take a key as another input

Symmetric Key Encryption



Symmetric Key Encryption (Cont.)

- Our goal
 - Ensuring confidentiality
- Three algorithms
 - $\text{KeyGen}() \rightarrow K$
 - $E(M, K) = E_K(M) = C$
 - $D(C, K) = D_K(C) = M$
- Correctness
 - For any $K, M, C = E_K(M)$
 - $D_K(C) = M$

Symmetric Key Encryption (Cont.)

- Attackers's goal: **any** knowledge about M_i in addition to its length or about K
 - Slightly better than 50% probability at guessing a single bit
 - Any information about the relationship between M_i and M_j
- Assumption: K is **randomly selected and secret**
- Recognition of success: **Eve** can generally tell if he has **correctly** and **fully** recovered M_i

Eve's Attacks

- Ciphertext only
 - Eve can observe every instance of C_i
 - Eve may also have partial information about M_i
 - Example: The language is probably English
- Known plaintext
 - Eve knows part of M_i and/or entire other M_j s
 - How could this happen?
 - Example: An encrypted HTTP GET request

Eve's Attacks

- Chosen plaintext
 - Eve gets Alice to send M_j s that Eve chooses
 - How could this happen?
 - Example: Eve spoofs a message from Alice's boss and asks her to forward to Bob
- Chosen ciphertext
 - Eve gets Bob to decrypt some C_j that Eve chooses
 - How could this happen?
 - Example: Bob receives C_j and needs to decrypt it to know the message. Eve can measure the time to decrypt.
- All of the above

Security of a Cryptosystem

- Crypto algorithms can be computationally secure
 - The cost of breaking the cipher is unacceptably high
 - The time to break the cipher exceeds the useful lifetime of the encrypted information
- Cost and time are difficult to estimate
- Exhaustive key space search (brute-force)
 - The cipher may be considered secure, if trying all possible keys take the most powerful computer that humans have ever built several billion years

A Simple Cipher

- Caesar Cipher
 - Rotating each letter by K positions
 - E.g., A $\rightarrow$ C, B $\rightarrow$ D, Y $\rightarrow$ A, if $K == 2$
- A more formal definition
 - Encode the 26 alphabets as integers in $[0, 25]$
 - $E(M, K) = M + K \bmod 26$
 - $D(C, K) = C - K \bmod 26$

Attacks against Ceasar Cipher

- Brute forcing
 - How much work is required to break it?
- Frequency analysis
- Known plaintext
 - Given pairs of (M, C) to derive the key
- Chosen plaintext
 - Choses a plaintext and observe the ciphertext

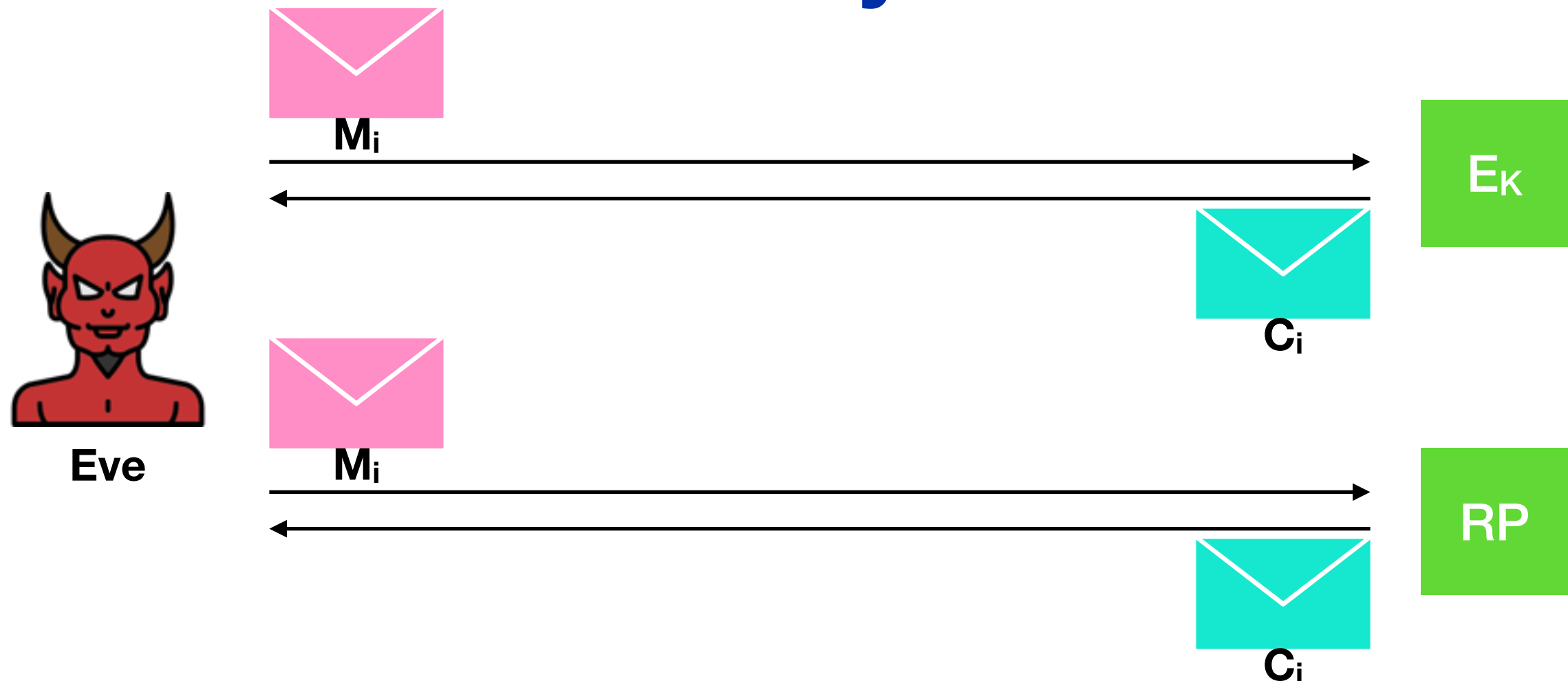
A Simple Cipher (Cont.)

- A better version
 - Rotating 1st character by K_1 , second by K_2 , ...
 - $K = \{K_1, K_2, \dots, K_n\}$
- Attacks
 - Brute forcing: How much work is required to break it?
 - Let $n = 4$
 - Frequency analysis
 - Need more ciphertext
 - Known plaintext and chosen plaintext
- A much better one?

Security of A Cryptosystem (Cont.)

- Kerckhoffs's principle
 - “A cryptosystem should be secure even if everything about the system, **except the key**, is public knowledge”
- Not enough to say that attacker cannot decrypt the ciphertext
- Attacker should not learn even **partial** information
- Attacker cannot **distinguish** which of 2 messages is encrypted in a **ciphertext**

A Security Game

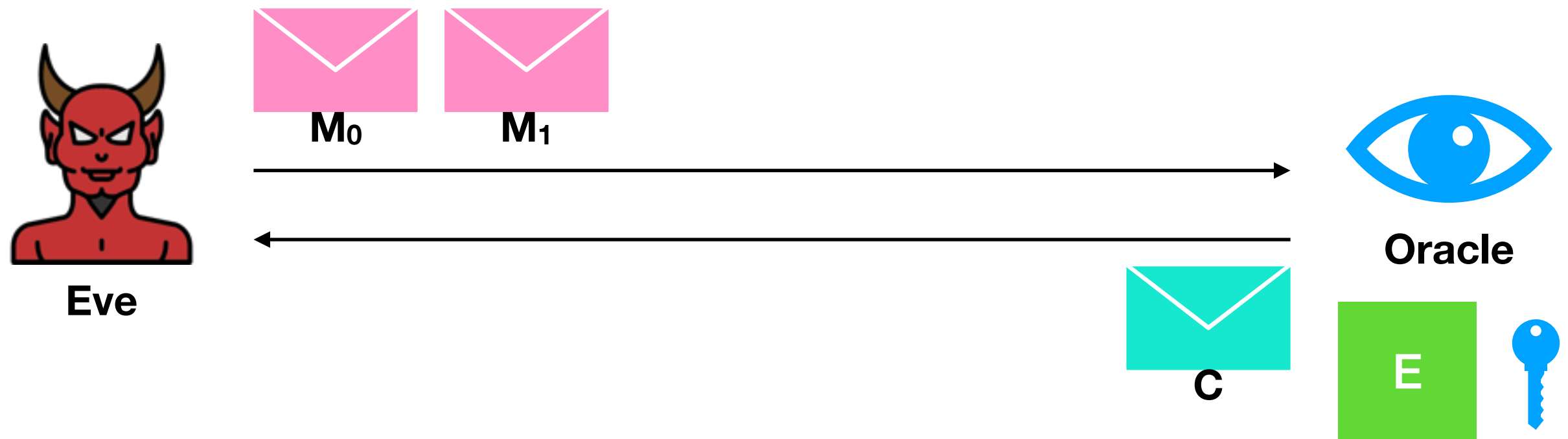


- Attacker is given two boxes, one for E_K and one for a random permutation RP
- Attacker can give input to each box and receive the output, for as many as times he/she wants
- Attacker guesses which box is E_K

Evaluating Security

- An encryption scheme meets the following condition
 - For any polynomial-time attacker, the probability to win this security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + e$, (e.g., $e = 1/2^{128}$)
- For an unknown key K , E_K behaves like a random permutation PR
 - The attacker cannot learn the plaintext given only the ciphertext (**some confidentiality**)
- The attacker may know that $M_i == M_j$, but not the values of M_i and M_j , if $E_K(M_i) == E_K(M_j)$

Indistinguishability under Known Plaintext Attack (IND-KPA) Security Game

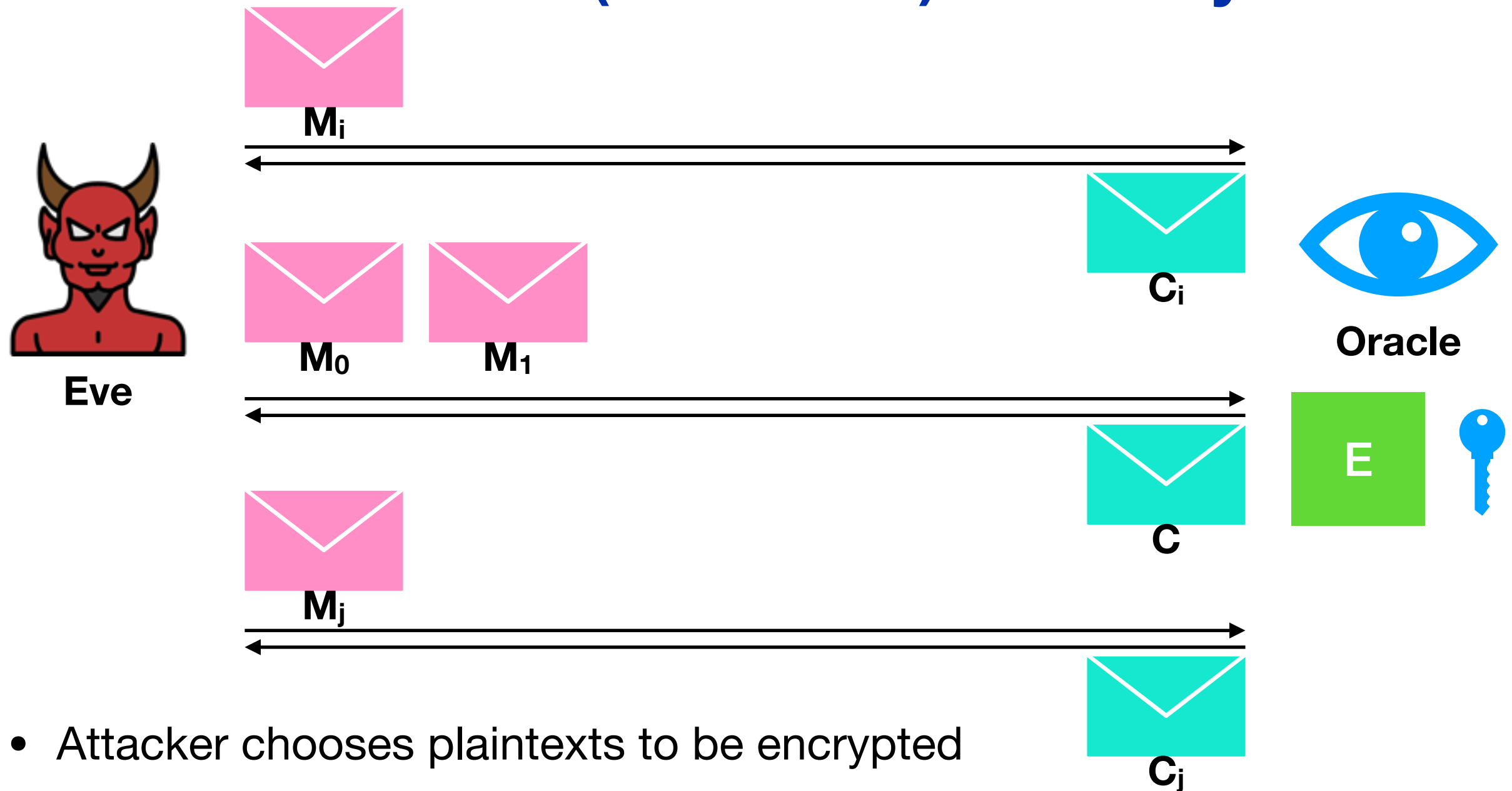


- Attacker chooses the two plaintexts
- Oracle randomly chooses a bit b and encrypts M_b
- Attacker guesses the value of b as b'
- Attacker wins iff $b == b'$

IND-KPA Security

- An encryption scheme is IND-kPA secure iff
 - For any polynomial-time attacker, the probability to win the IND-KPA security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + \epsilon$, (e.g., $\epsilon = 1/2^{128}$)
- IND-KPA security is not a very strong level of security guarantee

Indistinguishability under Chosen Plaintext Attack (IND-CPA) Security Game



- Attacker chooses plaintexts to be encrypted
- Attacker chooses two plaintexts; Oracle randomly encrypts one
- Attacker chooses other plaintexts to be encrypted
- Attacker guesses the value of b as b' , and wins iff $b == b'$

IND-CPA Security

- An encryption scheme is IND-CPA secure iff
 - For any polynomial-time attacker, the probability to win the IND-CPA security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + \epsilon$, (e.g., $\epsilon = 1/2^{128}$)
- IND-CPA secure is a very strong security guarantee
- Nothing leaks about the encrypted value other than its length
- IND-CPA security implies IND-KPA security

IND-CPA (Cont.)

- Are the following encryption schemes IND-CPA secure?
 - $E_K(M) = M \parallel M$
 - $E_K(M) = \text{random bits}$
 - $E_K(M) = (M+K) \bmod P$, where P is a prime number
- IND-CPA requires that the encryption scheme to be randomized
 - For the same plaintext, it outputs a different ciphertext each time

One-Time Pad

- A **random** key is chosen for **each** message M
- The key is as long as the message M

$$M_i \in \{0,1\}^n$$

$$K \in \{0,1\}^n$$

- $E(M, K) = M \wedge K$
- $D(C, K) = C \wedge K = (M \wedge K) \wedge K = M$
- The key is used only once

Security of OTP

- Shannon:
 - OTP is secure against one-time eavesdropping
 - The ciphertext reveals no information about the plaintext (except for its length)
- Claim: Given one ciphertext C , M_0 or M_1 is equally likely
 - $C = K_0 \wedge M_0$
 - $C = K_1 \wedge M_1$
 - ?

Security of OTP (Cont.)

- Is OTP IND-KPA secure?
- Proof idea:
 - Assumption: the adversary gets only one ciphertext
 - Show that $\Pr[M_0 \mid C]$ is $1/2$
- Is OTP IND-CPA secure?

One-Time Pad (Cont.)

- Problem #1: key generation
 - Need **truly** random, independent keys
- Problem #2: key distribution
 - Need a **secure** way to distribute the random key in each communication
 - If you can share the keys securely, why don't you just share the secret message?
- Problem #3: key length
 - Need the key as long as the message

Two-Time Pad

- What if we reuse a key?
 - $C_1 = E(M_1, K)$
 - $C_2 = E(M_2, K)$
- Problem?
 - $C_1 \wedge C_2 = ?$
 - $M_1 \wedge M_2$
- What does $M_1 \wedge M_2$ tells Eve?

How to Build an IND-CPA Secure Encryption Scheme?

- One-Time Pad is provably secure against one-time eavesdropping
- But it has several limitations
- Ingredients
 - One-Time Pad
 - Block Ciphers

Block Ciphers

- Building block:
 - A function $E: \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$
 - Fixing the key K : $E_K: \{0, 1\}^n \rightarrow \{0, 1\}^n$
 - $E_K(M) = E(M, K)$
- Properties
 - **Correctness**: E_K is a permutation (bijective)
 - **Efficiency**: Computation is very fast
 - **Security**: The permutation looks random without knowing K

Block Ciphers (Cont.)

- Data Encryption Standard (DES)
 - Designed in late 1970s
 - 56-bit key, 64-bit block
 - Tweaked by NSA (64-bit key -> 56-bit key)
 - Unbroken for decades
- Advanced Encryption Standard (AES)
 - 128, 196, or 256-bit key
 - 128-bit block
 - Not proven secure
 - No known flaw

Block Cipher Security

- For an unknown key K , E_K behaves like a random permutation PR
 - The attacker cannot learn the plaintext given only the ciphertext
- Block ciphers provide *some confidentiality*
- With the same key, the block cipher produces the *same* output on *same* inputs
- Are block ciphers IND-CPA secure?
 - No, because they are deterministic
 - How to win the IND-CPA game?

Using Block Ciphers

- We want two properties
 - Encrypt messages of arbitrary lengths
 - IND-CPA security even when reusing the same key to encrypt many messages
- Like OTP, block ciphers work on fixed-length input
 - M is shorter: padding
 - M is longer: applying the block cipher repeatedly
 - Called a **block cipher mode**
 - Can hide that same block is encrypted twice

Modes of Operation

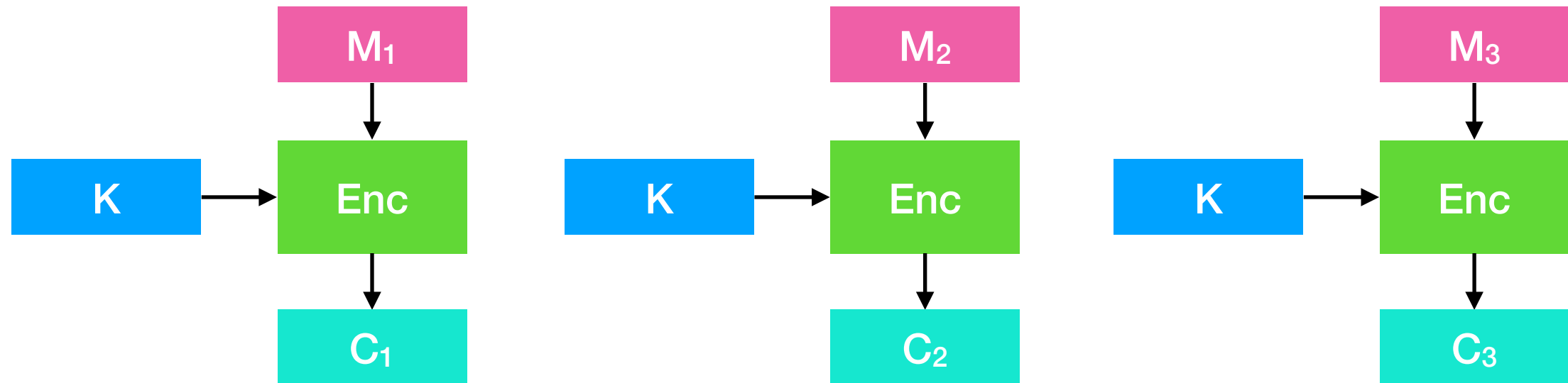
- Chain block ciphers in certain modes of operation
 - Invoke block cipher multiple times on inputs related to other blocks
- Need some **initial randomness**
 - IV (Initialization Vector)
- Why?
 - To prevent the encryption scheme from being deterministic

Electronic Code Book (ECB) Mode

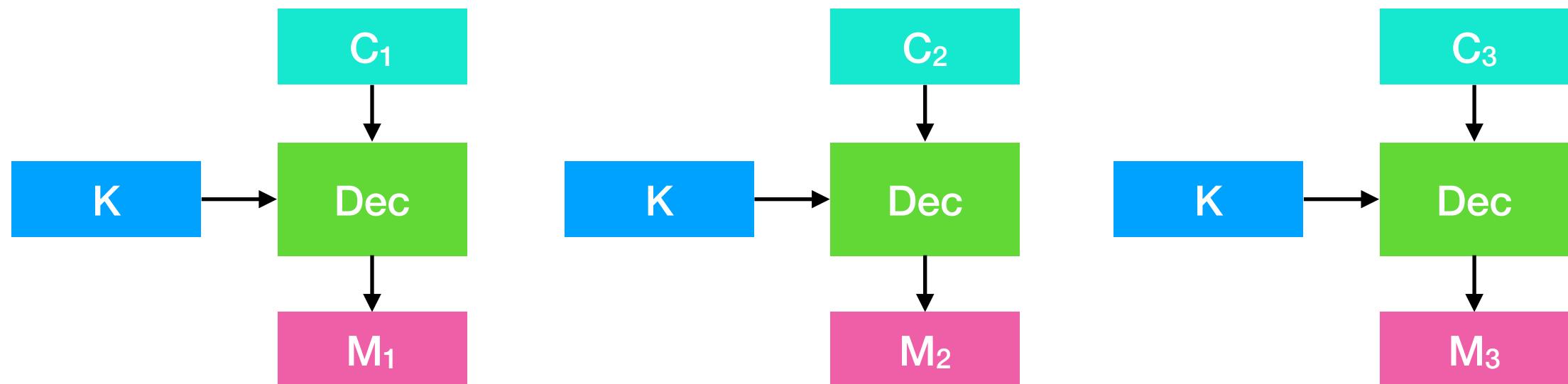
- Simplest mode
- Split a long message into n-bit blocks $M_1, M_2, \dots$
- Each block is encrypted/decrypted **independently**, using the **same key**
 - $C_i = E_k(M_i)$
 - $M_i = D_k(C_i)$
- Concatenate the encrypted/decrypted blocks to produce the output

ECB Mode (Cont.)

Encryption



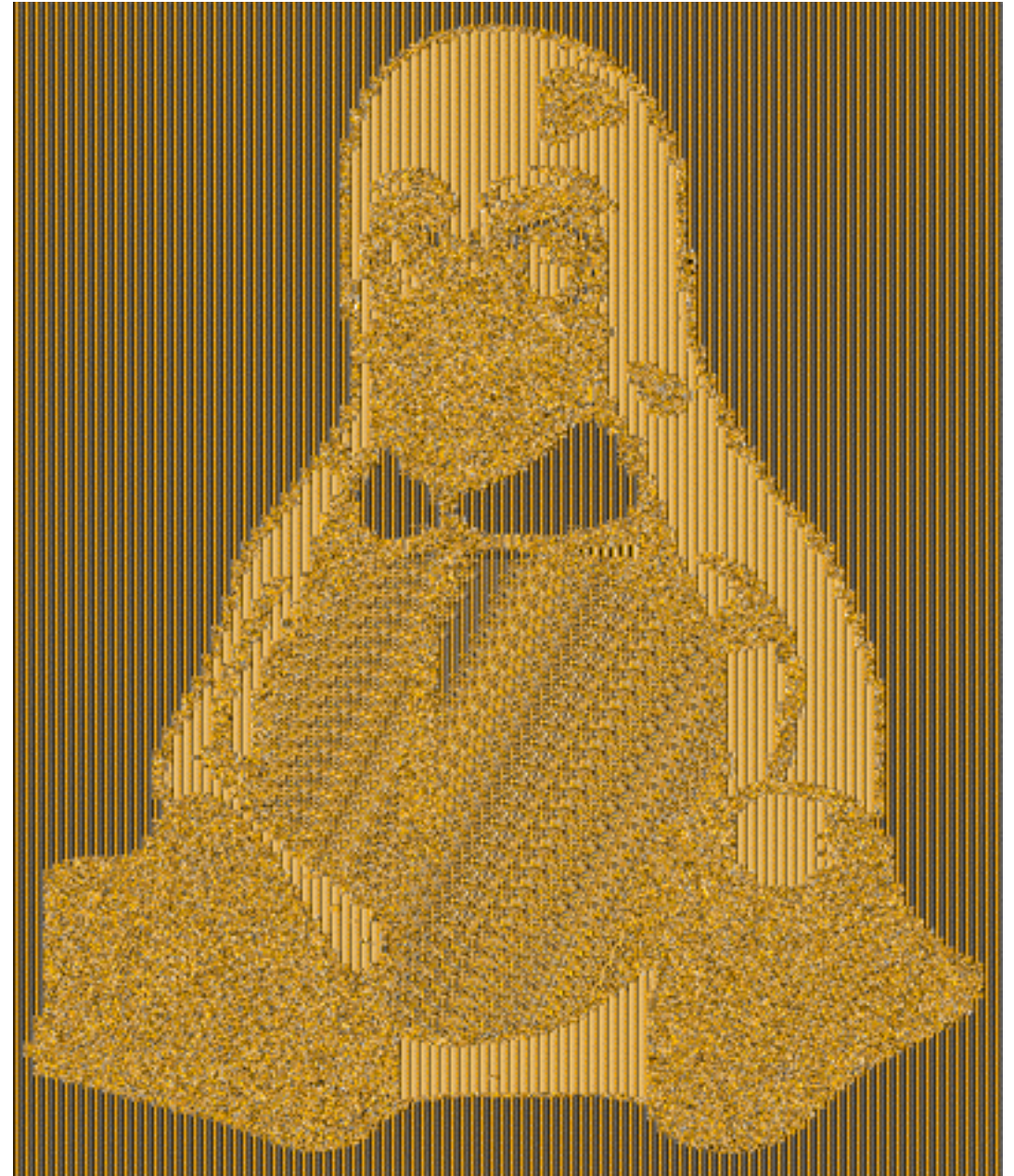
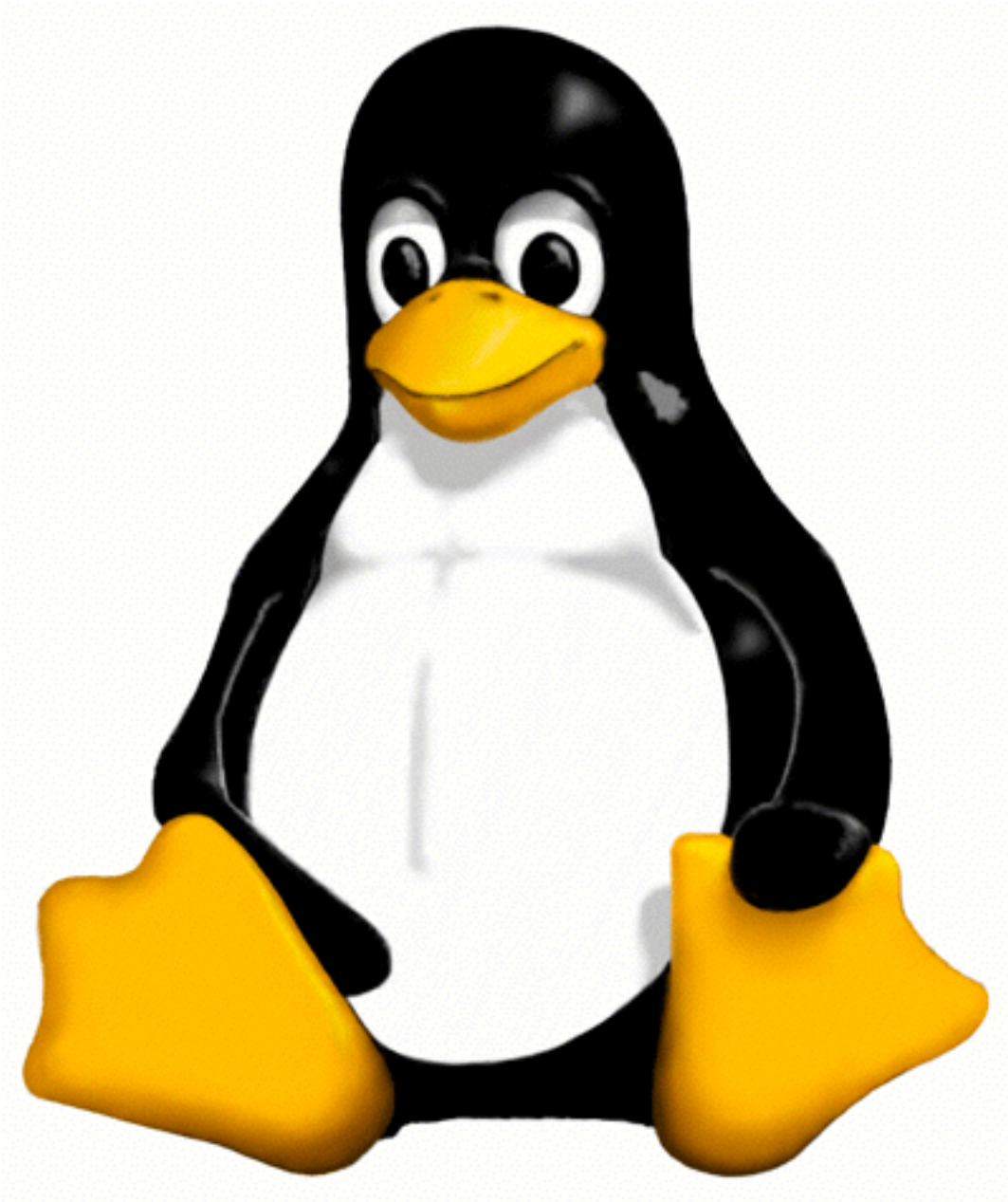
Decryption



Security of ECB Mode

- Is there any problem with ECB?
- Is ECB mode IND-CPA secure?

Security of ECB Mode (Cont.)

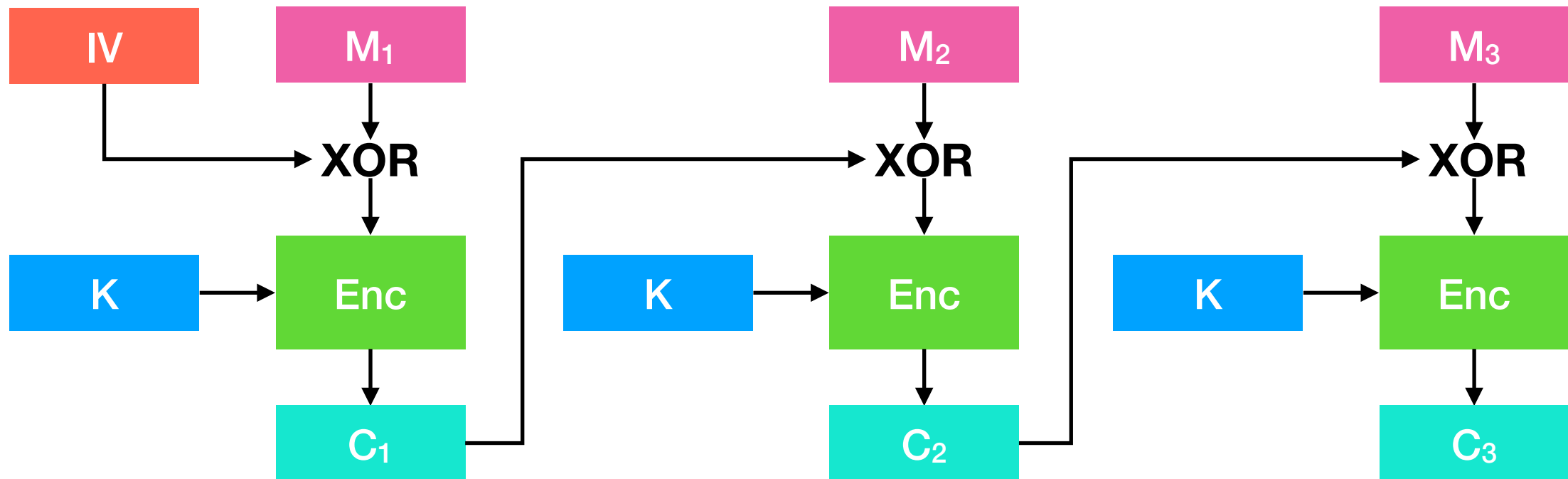


Cipher Block Chaining (CBC) Mode

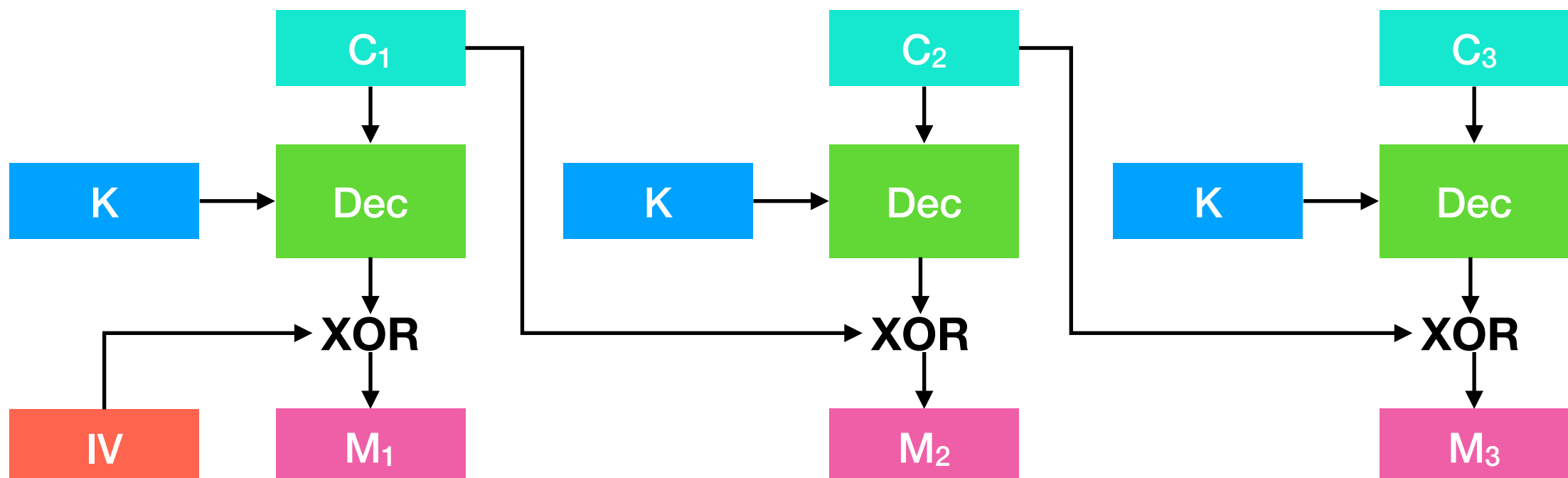
- A better mode: Including a varying, known nonce
- Split a long message into b -bit blocks $M_1, M_2, \dots$
- Choose a one-time random IV (Initialization Vector)
- Chain the block ciphers by including the output of the previous block as an input of the current block
- Final output is IV + concatenation of output blocks

CBC Mode (Cont.)

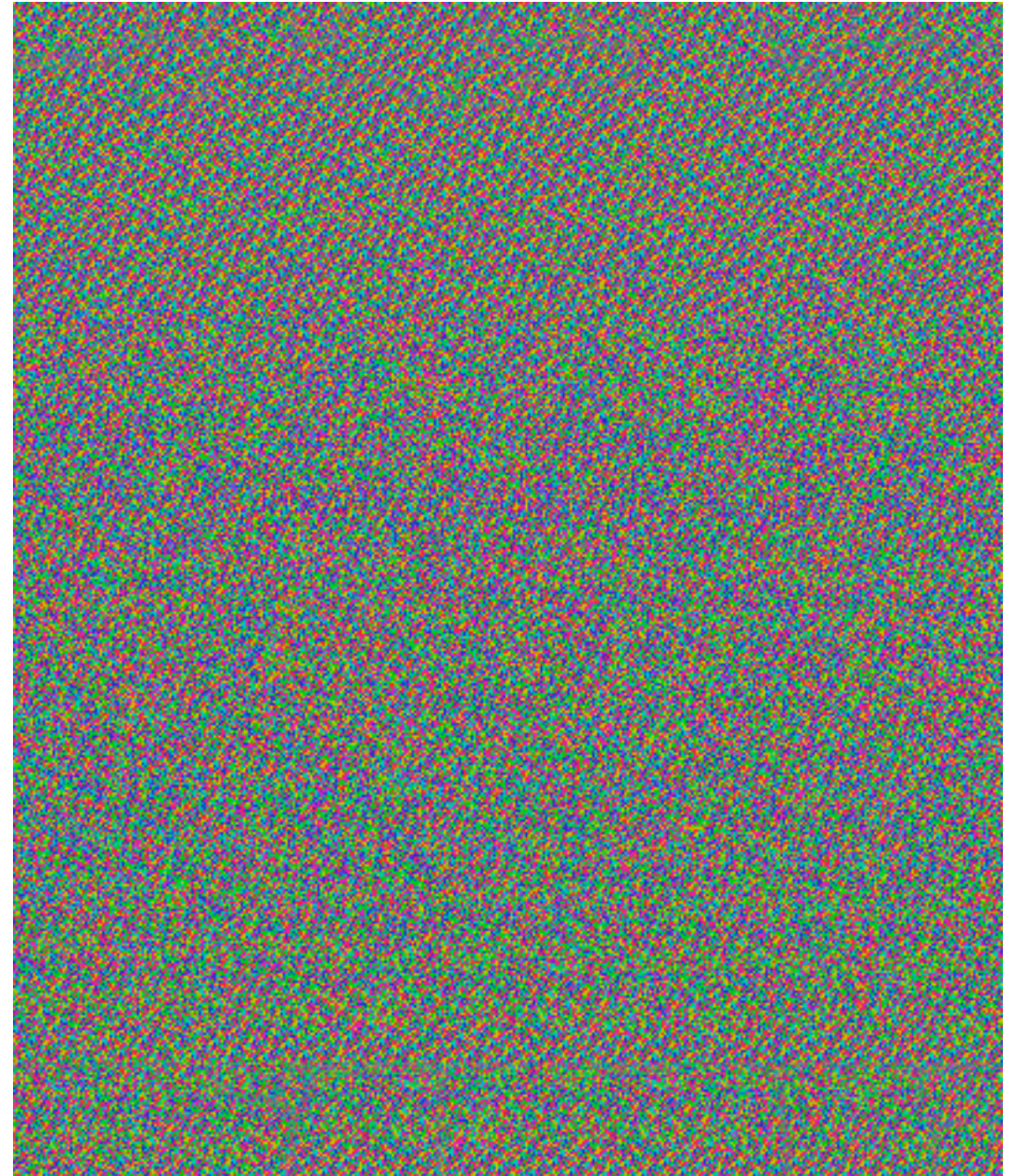
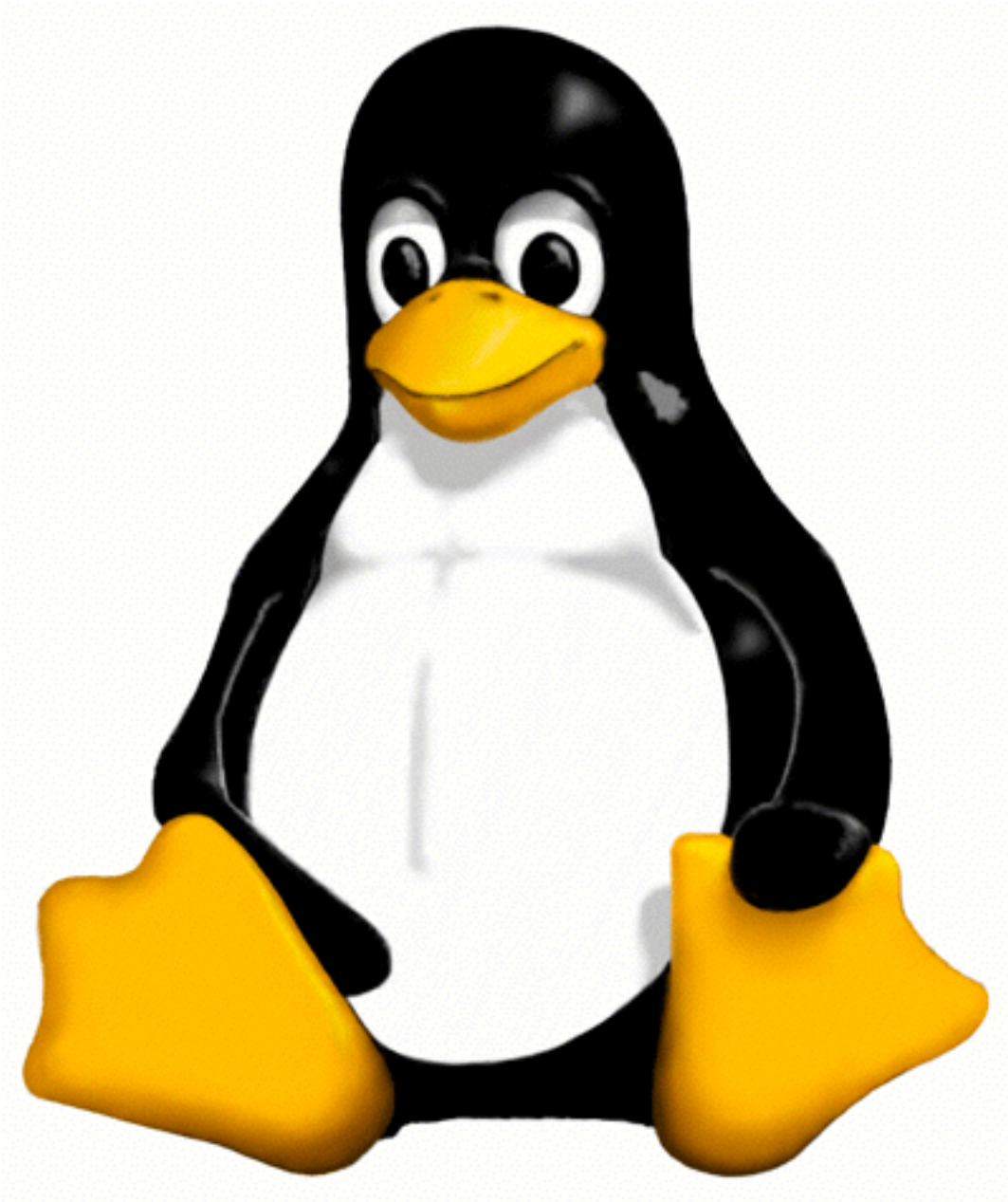
Encryption



Decryption



CBC Mode (Cont.)



CBC Mode (Cont.)

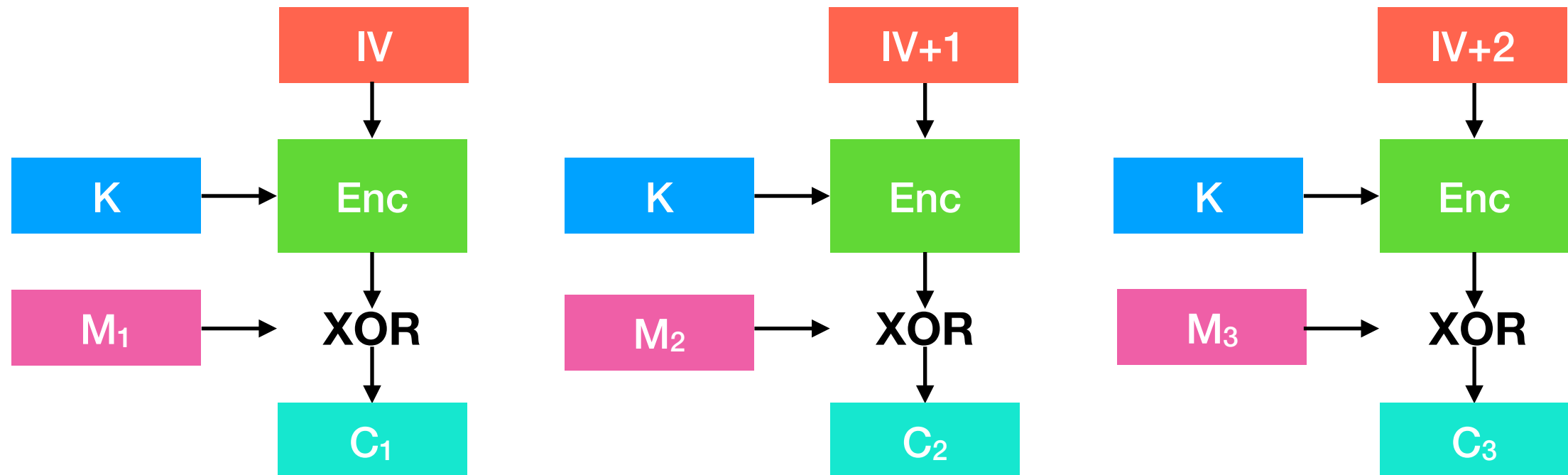
- Achieves IND-CPA security
- CBC mode is popular, still widely used
- Disadvantages
 - Sequential encryption
 - Hard to parallelize

Counter (CTR) Mode

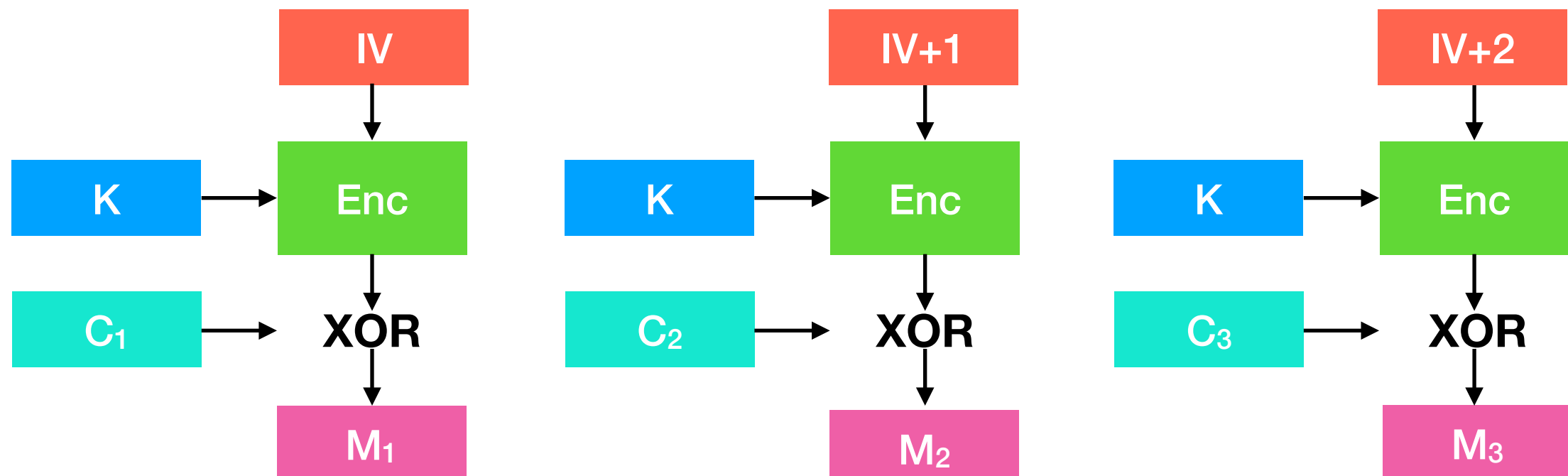
- CBC mode is widely used but hard to parallelize
- CTR mode
 - Parallelize encryption/decryption
 - Also provably secure
 - Select a random IV/nonce
 - Increment the IV for each block
- Note: CTR decryption uses block cipher's encryption, not decryption

CTR Mode (Cont.)

Encryption



Decryption



CBC vs. CTR

- Security
 - Both are IND-CPA, if no reuse of IV
- Performance
 - Both modes require the same amount of computation
 - CBC is parallelizable for only decryption
 - CTR is parallelizable for both encryption and decryption

Pseudorandom Number Generators

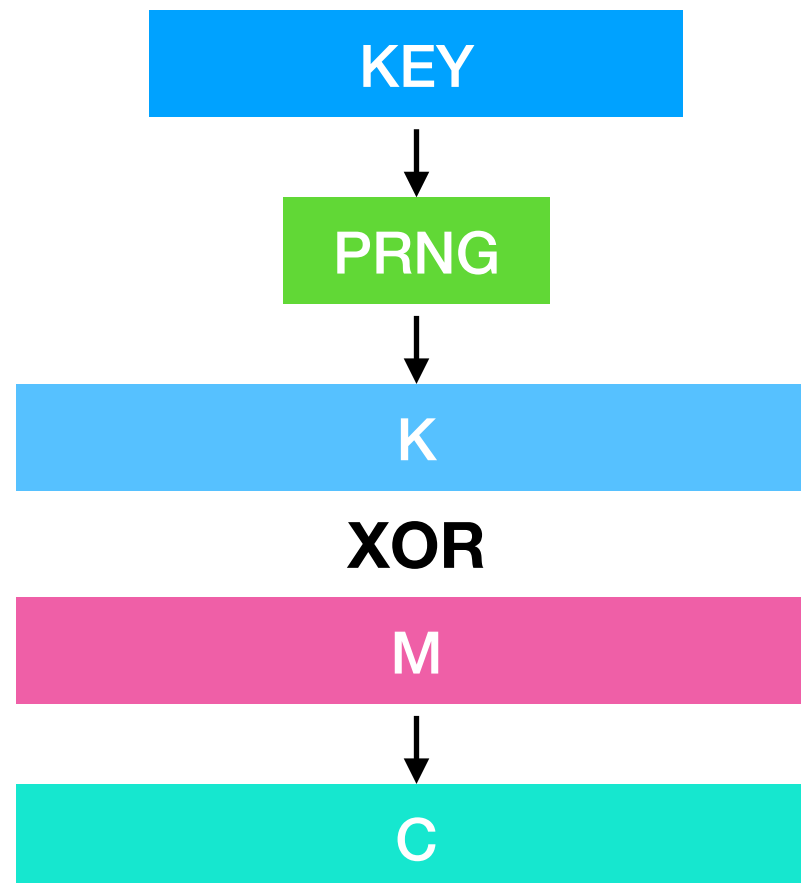
- Given a seed, outputs sequence of seemingly random numbers
- Can output arbitrarily many random bits
- Keep internal state
- Can a PRNG be truly random?
 - No
 - Why?

Pseudorandom Number Generators (Cont.)

- A cryptographically strong PRNG “looks” truly random to an attacker
 - Attacker cannot distinguish its output from a truly random sequence
- Generating randomness
 - `/dev/random`
 - Hardware interrupts
 - Hardware RNG: Intel RdRand instruction
- How to build a cryptographically strong PRNG?

Stream Ciphers

- Block cipher: fixed-size, stateless, requires modes to securely process longer messages
- Stream cipher:
 - Keep states from previous messages
 - Generate a one-time pseudorandom key K



Stream Ciphers (Cont.)

- Encryption
 - Choose a random IV
 - $E(M, K) = \text{PRNG}(K, \text{IV}) \text{ XOR } M$
- Decryption
 - $D(C, K) = \text{PRNG}(K, \text{IV}) \text{ XOR } C$
- Similar in spirit to OTP
- Can it encrypt message of any length?
 - PRNG can produce any number of random bits
- Can also encrypt a message multiple times

Symmetric Encryption Summary

- Desirable security is IND-CPA
- Block ciphers can be used to build IND-CPA secure encryption schemes with certain modes of operation
- Stream ciphers provide another way to encrypt
- Confidentiality is insufficient for most applications
- We need also to defend against active (tampering) attacks

Integrity and Authentication

- Why do we need integrity and authenticity?
 - Active attacker
 - Can inject new messages (ciphertext)
 - Can replay previous cipher texts
 - Can cause messages to be reordered or discarded
 - Man-in-the-middle attacker
 - Can be much more powerful than just eavesdropping

Integrity and Authentication (Cont.)

- Goals:
 - Given a message, prove it is not altered
 - Given a message, prove it is indeed sent from the sender
- Approach: Providing integrity and authentication with cryptographically strong hash functions
- Examples:
 - MD5: 128 bits, broken
 - SHA-1: 160 bits, broken
 - SHA-256: 256 bits, not broken yet

Cryptographically Strong Hash Functions

- Properties required
 - One-way
 - Given y , difficult to find x s.t. $\text{Hash}(x) = y$
 - Collision resistance
 - Difficult to find x and x' s.t. $\text{Hash}(x) = \text{Hash}(x')$
 - Second pre-image resistance
 - Given x , difficult to find x' s.t. $\text{Hash}(x) = \text{Hash}(x')$

SHA-256

- SHA-256/SHA-384 are two parameters for the SHA-2 hash algorithm, returning 256-bit or 384-bit hashes
 - Can work on inputs of arbitrary length
- Is vulnerable to a length-extension attack
 - S is secret
 - Attacker knows $\text{len}(S)$, $H(S)$
 - Attacker can calculate $H(S||M)$ for an M
 - Works because all the internal state at the point calculating $H(S||\dots)$ can be derived from $H(S)$ and $\text{len}(S)$
- SHA-3 standard does not have this property

Message Authentication Code

- Cryptographically strong hash functions are not sufficient for integrity and authentication
 - Attackers can also compute a valid hash value
- Message Authentication Code (MAC) provides message integrity and authentication
 - It does not provide confidentiality
 - Needs a secret shared key K
 - The message itself can be in plaintext
- Idea: Compute and verify a tag of the message that only Alice and Bob can produce

MAC (Cont.)

- Alice
 - Computes $T = \text{MAC}(M, K)$
 - Sends $\{M, T\}$
- Bob
 - Receives $\{M', T'\}$
 - Computes $T'' = \text{MAC}(M', K)$ and compare T' to T''
- Integrity is provided if the attacker cannot
 - Find M^* and T^* s.t. $T^* = \text{MAC}(M^*, K)$
 - Find M^* s.t. $T = \text{MAC}(M^*, K)$
- Cryptographically strong hash functions + secret shared key

Hash-MAC (HMAC)

- Most widely used MAC on the Internet
- Building secure MAC
 - H is a cryptographically strong hash function
 - Pad_i , Pad_o are publicly known strings
 - K is a secret key
- $\text{HMAC}(M, K) = H(K^{\wedge}\text{Pad}_o \parallel H(K^{\wedge}\text{Pad}_i \parallel M))$
- Believed to be secure even if H is not cryptographically strong

Using MAC

- MAC in general do not provide any guarantee on confidentiality of the message
- It is recommended to compute MAC
 - Using a separate key
 - On the ciphertext
- K_E : Encryption key; K_I : MAC Key
- Option 1 (SSL): $C = \text{Enc}(M \parallel \text{MAC}(M, K_I), K_E)$
- Option 2 (SSH): $C = \text{Enc}(M, K_E) \parallel \text{MAC}(M, K_I)$
- Option 3 (IPsec): $C = \text{Enc}(M, K_E) \parallel \text{MAC}(\text{Enc}(M, K_E), K_I)$

Symmetric Cryptography Summary

- Pros:
 - Confidentiality: Strong
 - Integrity: Weak
 - Authenticity: Weak
- Cons:
 - Secret key sharing is difficult, typically requiring another secure communication channel
 - Need more keys if more parties are involved
 - Cannot prove that the message is sent/encrypted by one of the involved parties

Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh, Giovanni Vigna, Raluca Ada Popa and Nicholas Weaver